Indexing

We know that data is stored in the form of records. Every record has a key field, which helps it to be recognized uniquely.

Indexing is a data structure technique to efficiently retrieve records from the database files based on some attributes on which the indexing has been done. Indexing in database systems is similar to what we see in books.

Index structure:

Indexes can be created using some database columns.

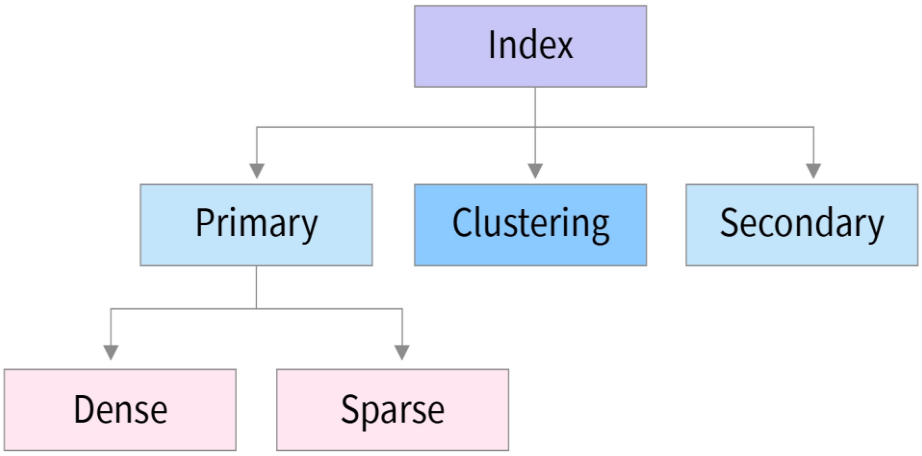
Search key	Data Reference

Fig: Structure of Index

- The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Types of Indexing:

Indexing is defined based on its indexing attributes. Indexing can be of the following types –



Single Level Indexing

Single Level Indexing is further divided into three categories:

1. Primary Indexing
2. Secondary Indexing
3. Cluster Indexing

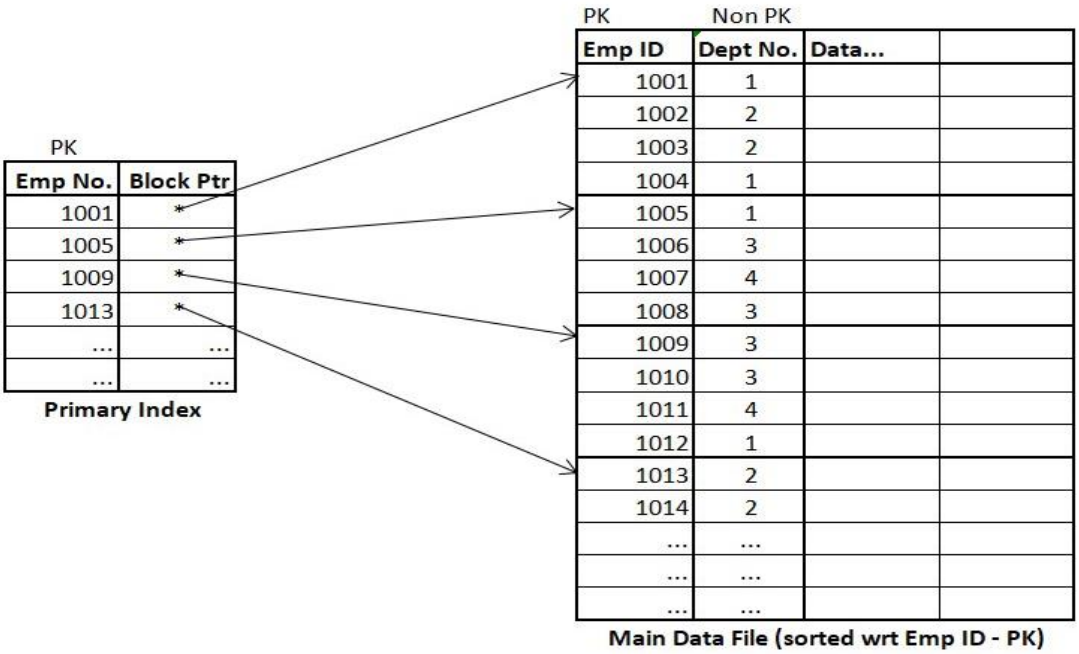
Primary Indexing

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.

Characteristics of Primary Indexing:

- Search Keys are unique.
- Search Keys are in sorted order.
- Search Keys cannot be null as it points to a block of data.
- Fast and Efficient Searching.

Example:



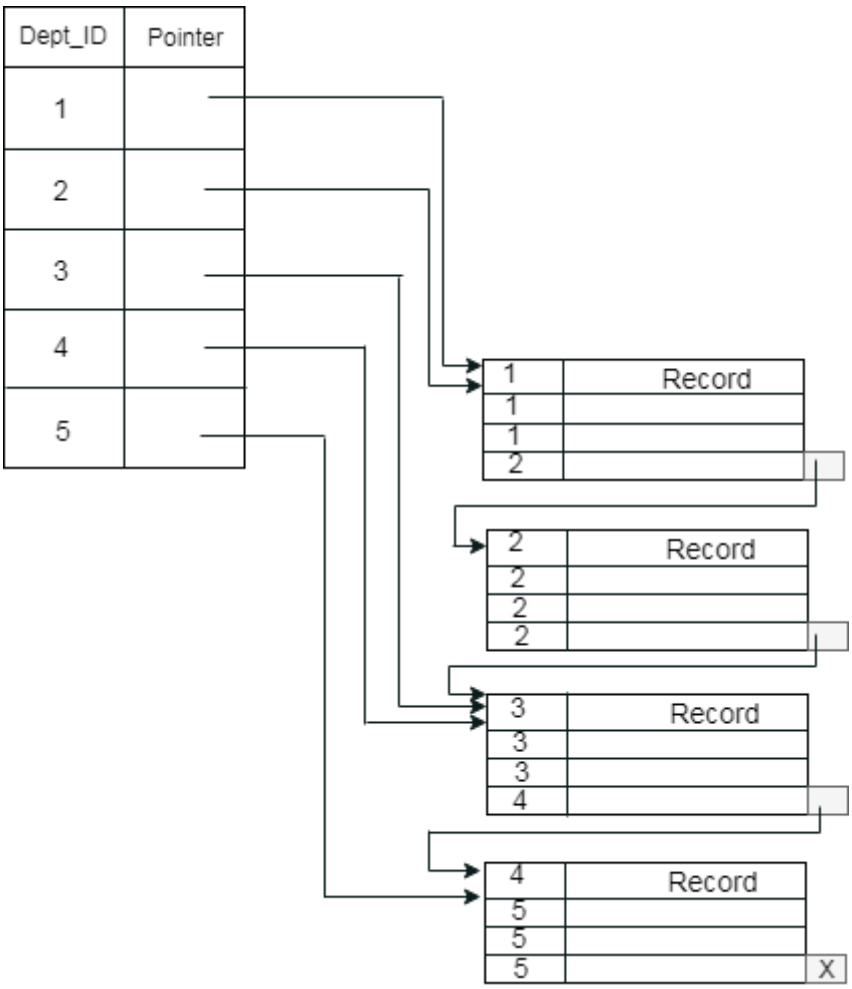
Cluster Indexing

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

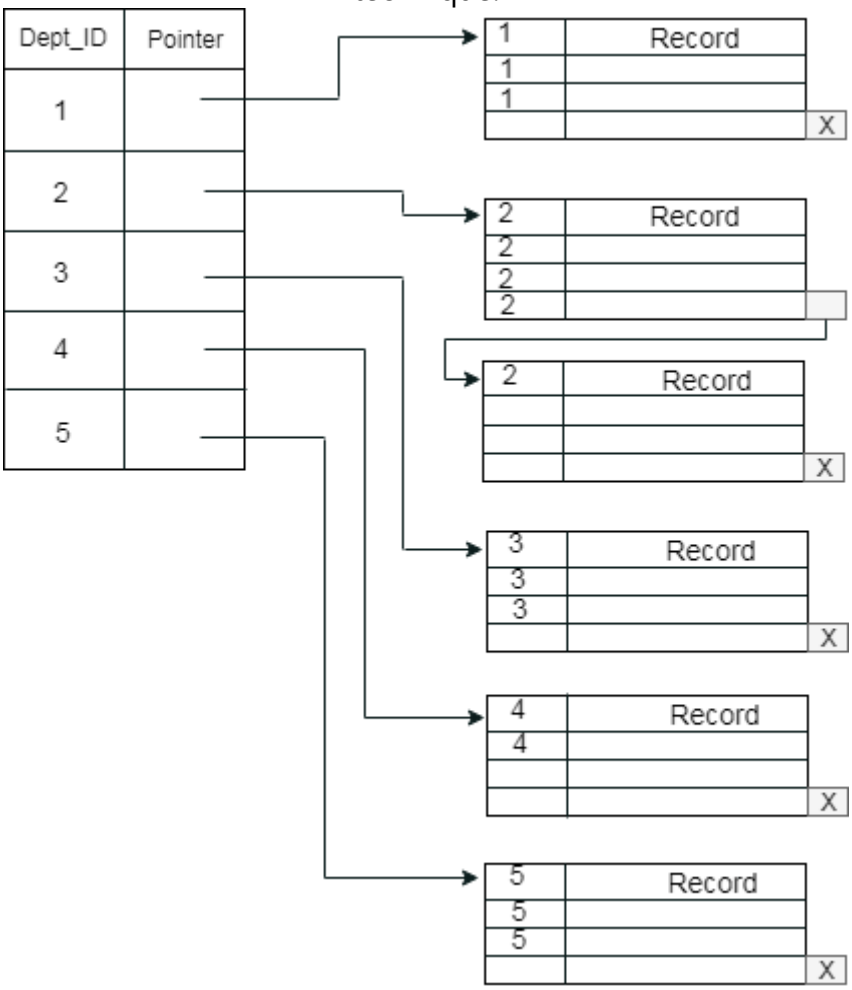
Characteristics of Clustered Indexing:

- Search Keys are non-key values.
- Search Keys are sorted.
- Search Keys cannot be null.
- Search Keys may or may not be unique.
- Requires extra work to create indexing.

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



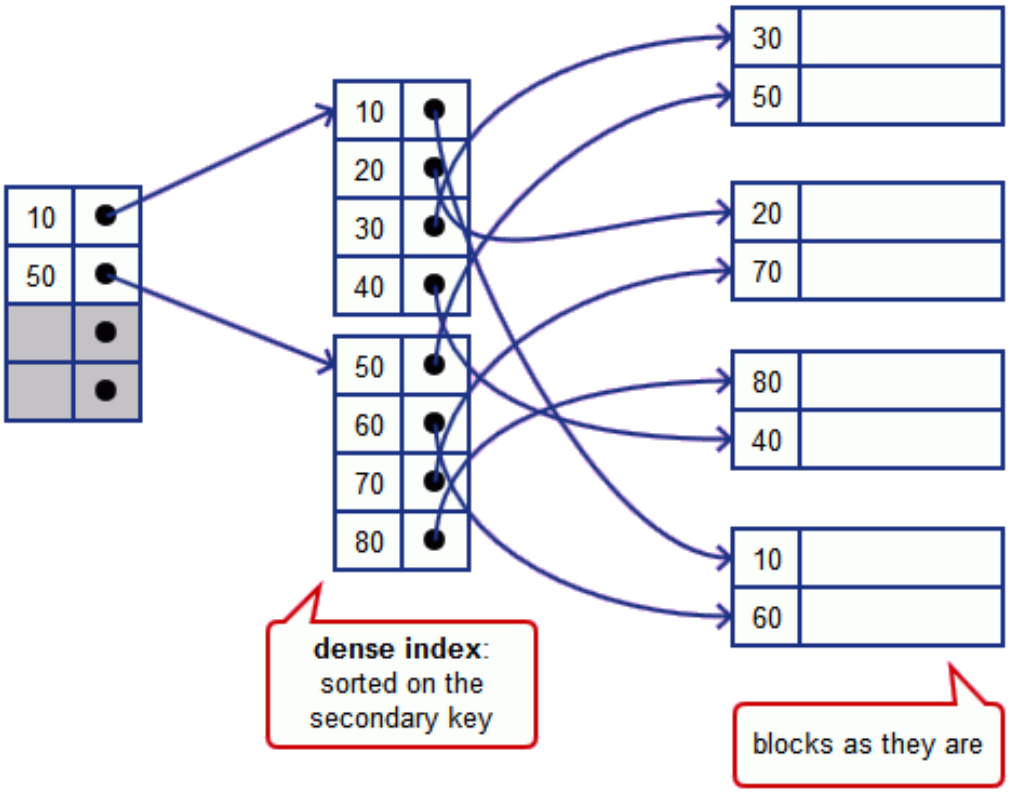
Secondary Indexing

It is a two-level indexing technique used to reduce the mapping size of the primary index. The secondary index points to a certain location where the data is to be found but the actual data is not sorted like in the primary indexing. Secondary Indexing is also known as non-clustered Indexing.

Characteristics of Secondary Indexing:

- Search Keys are Candidate Keys.
- Search Keys are sorted but actual data are unsorted or unordered.
- Requires more time than primary indexing.
- Search Keys cannot be null.
- Faster than clustered indexing but slower than primary indexing.

Example:



Ordered Indexing:

Ordered indexing is the traditional way of storing that gives fast retrieval. The indices are stored in a sorted manner hence it is also known as ordered indices.

Ordered Indexing is further divided into two categories:

Dense Index

- The dense index contains an index record for every search key value in the data file. It makes searching faster.
- In this, the number of records in the index table is same as the number of records in the main table.
- It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.

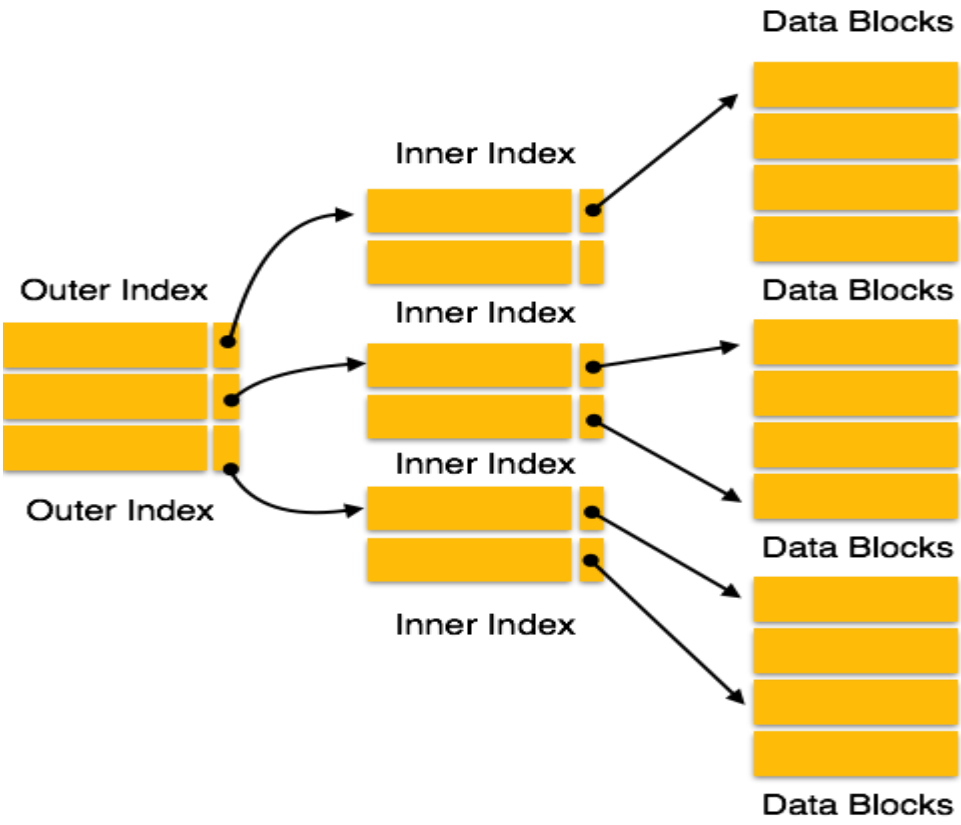
Sparse Index

- In the data file, index record appears only for a few items. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.
- If the data we are looking for is not where we directly reach by following the index, then the system starts sequential search until the desired data is found.

Multi-Level Indexing:

Since the index table is stored in the main memory, single-level indexing for a huge amount of data requires a lot of memory space. Hence, multilevel indexing was introduced in which we divide the main data block into smaller blocks. This makes the outer block of the index table small enough to be stored in the main memory.

Example:



We use the B+ Tree data structure for multilevel indexing. The leaf nodes of the B+ tree contain the actual data pointers. The leaf nodes are themselves in the form of a linked list. This linked list representation helps in both sequential and random access.

Indexing at a glance

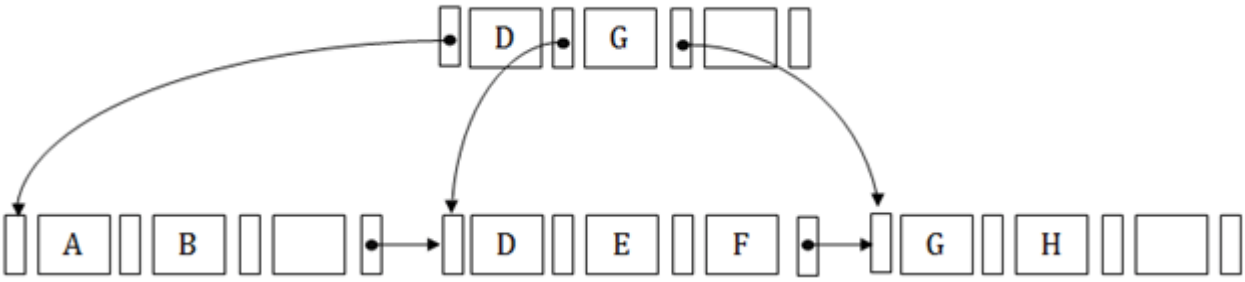
Ordered file	Primary Index (Sparse)	Clustered Index (Sparse)
Unordered file	Secondary Index	Secondary Index
	Key	Non-Key

B+ Tree

- The B+ tree is a balanced binary search tree. It follows a multi-level index format.
- In the B+ tree, leaf nodes denote actual data pointers. B+ tree ensures that all leaf nodes remain at the same height.
- In the B+ tree, the leaf nodes are linked using a link list. Therefore, a B+ tree can support random access as well as sequential access.

Structure of B+ Tree

- In the B+ tree, every leaf node is at equal distance from the root node. The B+ tree is of the order n where n is fixed for every B+ tree.
- It contains an internal node and leaf node.



Internal node

- An internal node of the B+ tree can contain at least n/2 record pointers except the root node.
- At most, an internal node of the tree contains n pointers.

Leaf node

- The leaf node of the B+ tree can contain at least n/2 record pointers and n/2 key values.
- At most, a leaf node contains n record pointer and n key values.
- Every leaf node of the B+ tree contains one block pointer P to point to next leaf node.

B^+ Tree Insertion

1. Every element is inserted into a leaf node. So, go to the appropriate leaf node.
2. Insert the key into the leaf node in increasing order only if there is no overflow. If there is an overflow go ahead with the following steps

Case 1: If a leaf node overflows

- Split the leaf node into two nodes.
- The first node contains the first **ceil((m-1)/2)** keys.
- The second node contains the remaining keys.
- Copy the smallest key from the second node (i.e., the first key of the second node) to the parent node.**(Right biased)**

Case 2: If a non-leaf node overflows

- Similar to the leaf node, split the non-leaf node into two nodes.
- The first node contains the first **ceil((m-1)/2)** keys.
- Move the smallest key from the second node to the parent.
- The second node contains the remaining keys.

B+ Tree Deletion

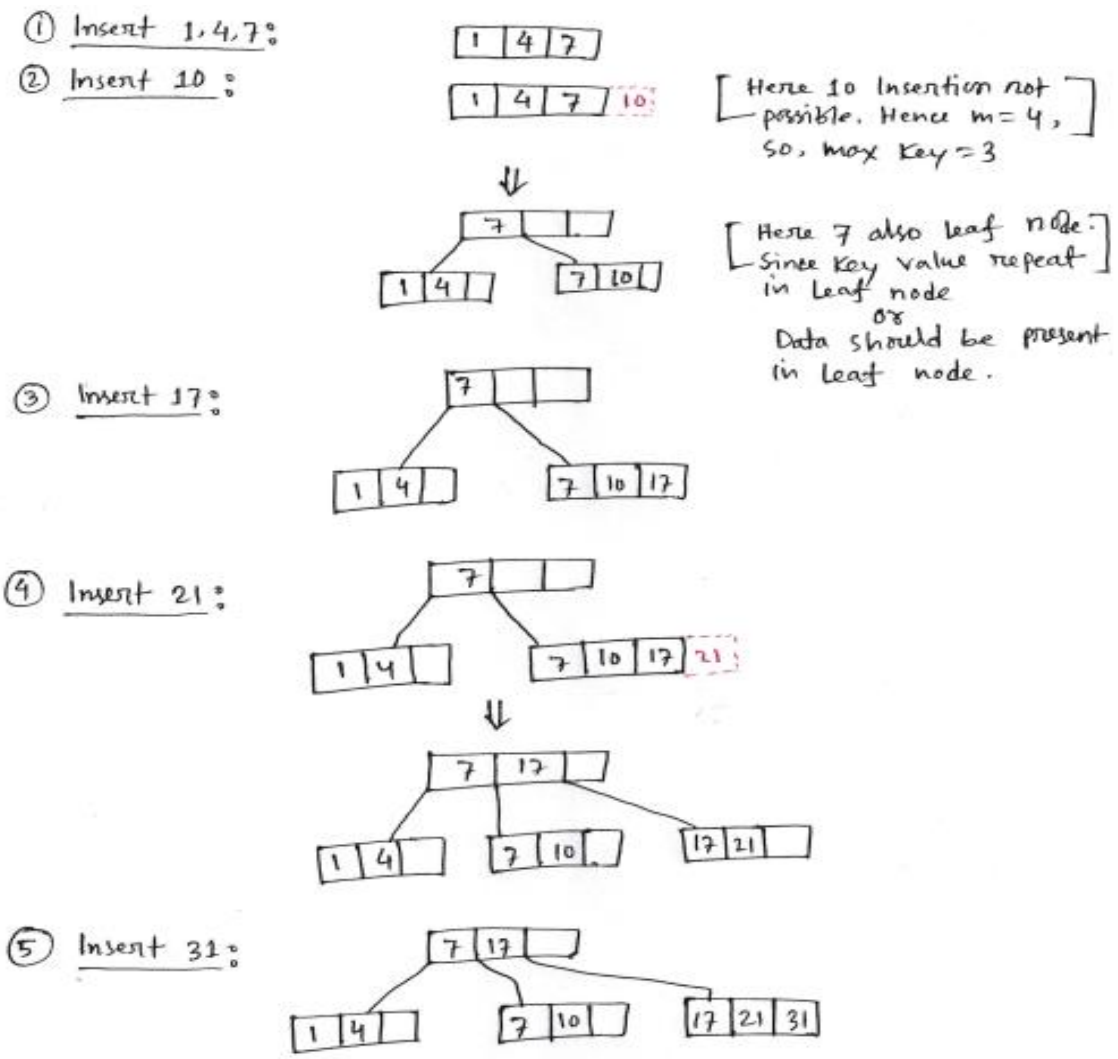
- B^+ tree entries are deleted at the leaf nodes.
- The target entry is searched and deleted.
    - If it is an internal node, delete and replace it with the entry from the left position.
- After deletion, underflow is tested,
    - If underflow occurs, distribute the entries from the nodes left to it.
- If distribution is not possible from the left, then
    - Distribute from the nodes right to it.
- If distribution is not possible from left or from right, then
    - Merge the node with left and right to it.

B+ Tree Creation

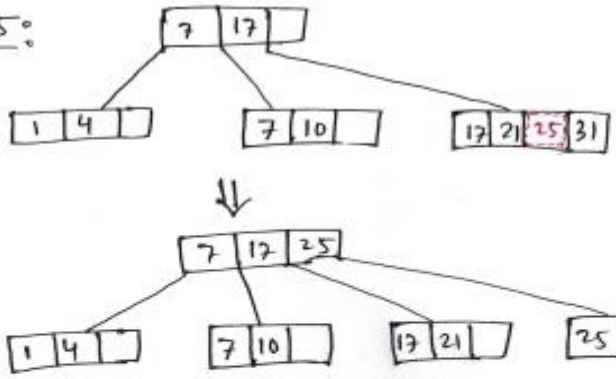
Suppose you are given some keys are 1,4,7,10, 17,21,31,25,19,20, 28,42 and order m=4. Create a B+ tree for the following keys.

Solution:

Hence Order (m)=4
Max Children=4
Min Children=$\lceil\frac{m}{2}\rceil = 2$
Max Keys=(m-1) =3
Min keys=$\left\lceil\frac{m}{2}\right\rceil - 1 = 1$

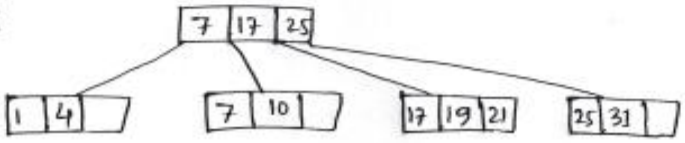


⑥ Insert 25:

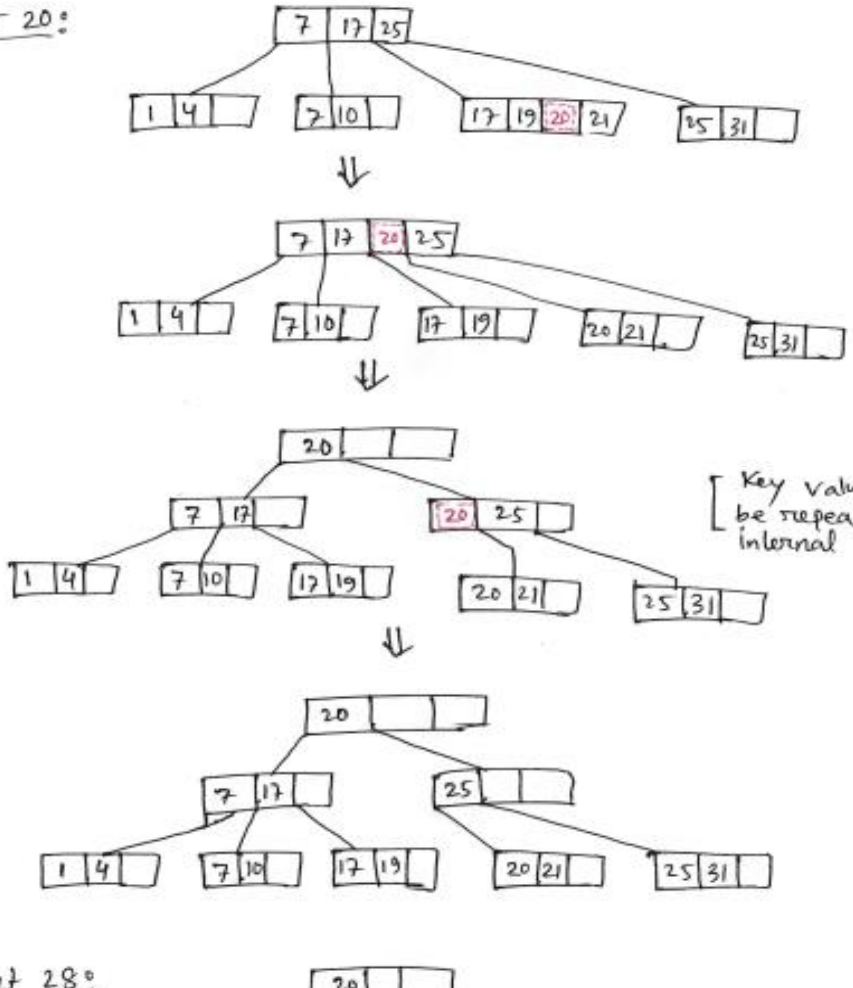


[25 Insertion not possible]

⑦ Insert 19:

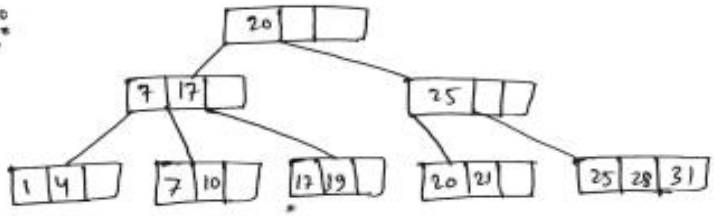


⑧ Insert 20:

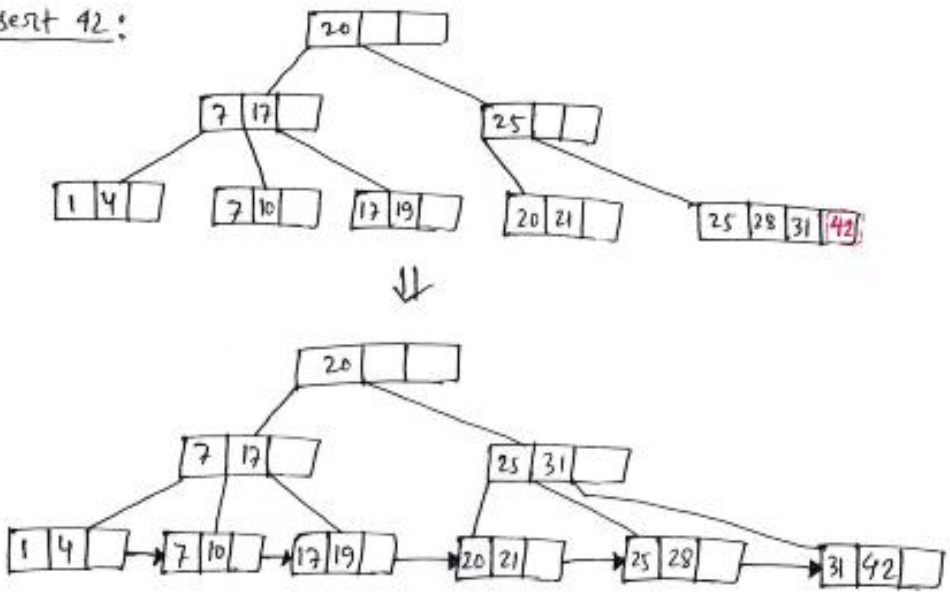


[Key value cannot be repeat in internal node]

⑨ Insert 28:

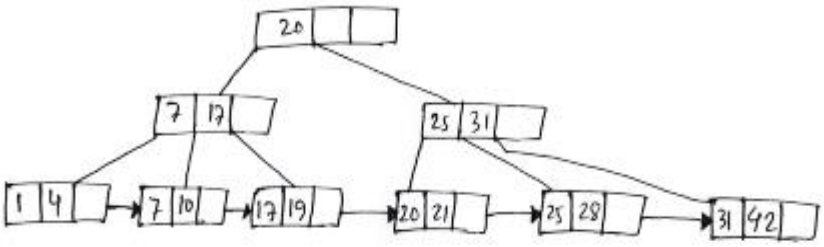


(10) Insert 42:


20
7 17
25
1 4
7 10
17 19
20 21
25 28 31 42
20
7 17
25 31
1 4
7 10
17 19
20 21
25 28
31 42

B+ Tree Deletion

Suppose you are given a B+ Tree that shown in the figure.

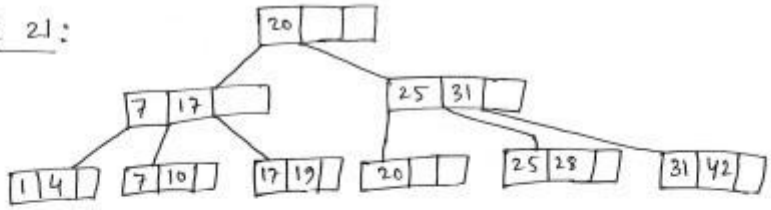


From the B+ tree delete: 21, 31, 20, 10, 7, 25, 42, 4

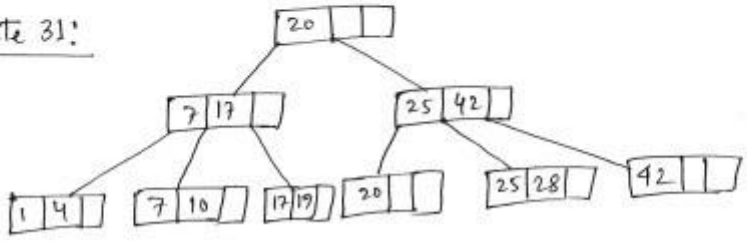
Solution:

Hence Order (m)=4
Max Children=4
Min Children=$\lceil\frac{m}{2}\rceil = 2$
Max Keys=(m-1) =3
Min keys=$\left\lceil\frac{m}{2}\right\rceil - 1 = 1$

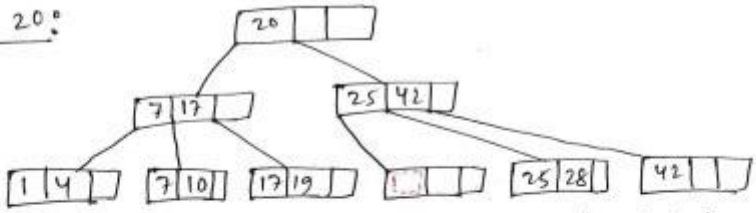
① Delete 21:



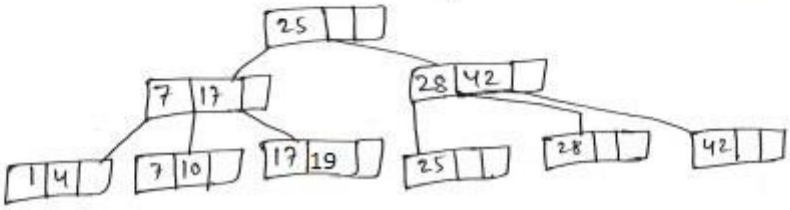
② Delete 31:



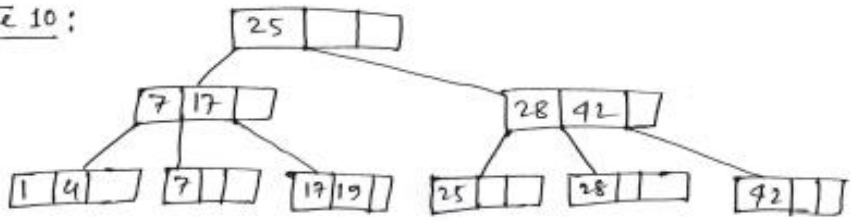
③ Delete 20:



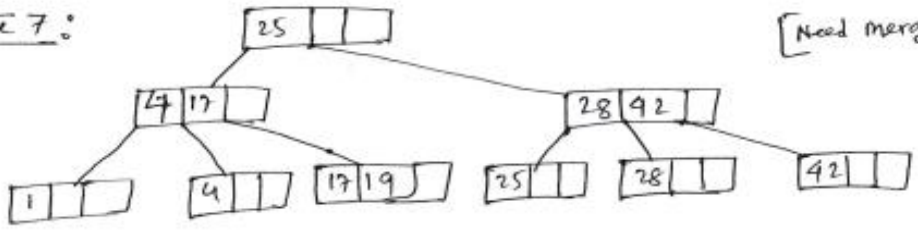
Hence, leaf node having zero keys and its not possible. So, borrow from left or right siblings. Immediate left or right node.



④ Delete 10 :

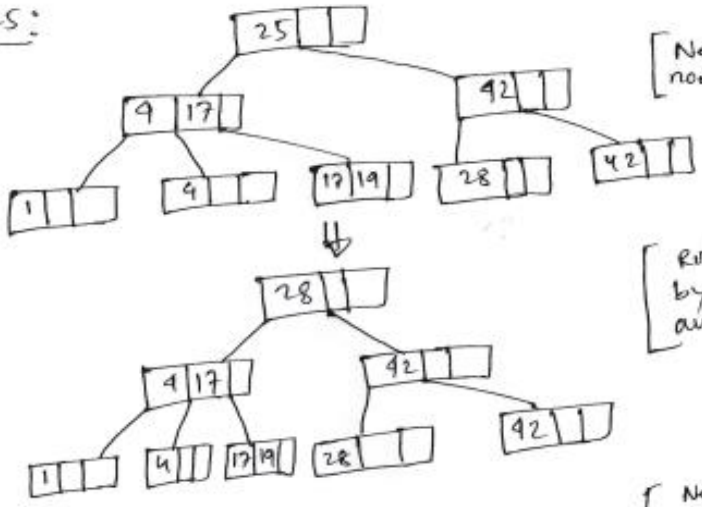


⑤ Delete 7 :



[Need merge]

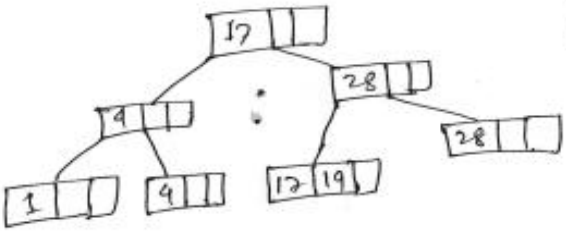
⑥ Delete 25 :



[Need merge TWO nodes □ & 28 = 28]

[Root 25 Replaced by min leaf value and its 28]

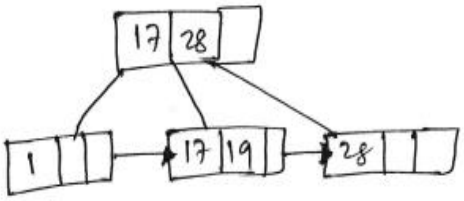
⑦ Delete 42 :



[Need merge TWO nodes □ & 28 = 28]

[Minimum children should be 2 and 17 would be passed to Root]

⑧ Delete 4 :



[Need merge two nodes □ & 1 = 1 and hence min children = 2. So 28 need merge with Root/Parents.]

Advantages of Indexing

- Indexing helps in faster query results or quick data retrieval.
- Indexing helps in faster sorting and grouping of records
- Some Indexing uses sorted and unique keys which helps to retrieve sorted queries even faster.
- Index tables are smaller in size so require lesser memory.
- As Index tables are smaller in size, they are stored in the main memory.
- Since CPU speed and secondary memory speed have a large difference, the CPU uses this main memory index table to bridge the gap of speeds.
- Indexing helps in better CPU utilization and better performance.

Disadvantages of Indexing

- Indexing necessitates more storage space to hold the index data structure, which might increase the total size of the database.
- **Increased database maintenance overhead:** Indexes must be maintained as data is added, destroyed, or modified in the table, which might raise database maintenance overhead.
- Indexing can reduce insert and update performance since the index data structure must be updated each time data is modified.
- **Choosing an index can be difficult:** It can be challenging to choose the right indexes for a specific query or application and may call for a detailed examination of the data and access patterns.

Conclusion

- Indexing is a technique that uses data structures to optimize the searching time of a database query.
- Index table contains two columns namely Search Key and Data Pointer or Reference.
- There are three types of indexing namely Ordered, Single-level, and multi-level.
- Single Level Indexing is divided into three types namely Primary (index table is created using primary keys), Secondary (index table is created using candidate keys), and Clustered (index table is created using non-key values).
- Ordered Indexing is divided into two types namely dense (index table contains records for every search key value of the database) and sparse (do not include search key for every record).
- Multi-level Indexing uses B+ Trees to store data pointers.
- Indexing helps in faster data retrieval and better performance.
- Most commonly used indexing attributes are: Standard (B-trees) and Bitmaps, Ascending and Descending attribute, Column and Functional attribute, Single-Column, and Concatenated attribute, Non-partitioned and Partitioned attribute.

Useful Links

Why Indexing is used?
https://www.youtube.com/watch?v=E--yzX05_k8

Primary Index in DBMS with Example
https://www.youtube.com/watch?v=4E-MGnjMhRw

Clustered Index in DBMS with Example
https://www.youtube.com/watch?v=UpJ9ICmzaAM

Secondary Index in Database with Example
https://www.youtube.com/watch?v=atfF_QJ7a8M

B Trees and B+ Trees. How they are useful in Databases
https://www.youtube.com/watch?v=aZjYr87r1b8&t=1s

B+ Tree Creation example
https://www.youtube.com/watch?v=DqcZLulVJ0M

B+ Tree Deletion example
https://www.youtube.com/watch?v=pGOdeCpuwpI

B+ Tree Visualization Website
https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html